

Réduire la consommation de tokens sans perdre en qualité

RTK : Filtrage en Amont

RTK (Rust Token Killer) intercepte les outputs de commandes **avant** qu'ils entrent dans le contexte de Claude. Ce n'est pas un résumé post-hoc, c'est un filtre qui élimine le bruit structurellement redondant.

```
# Installation brew install rtk-ai/tap/rtk
# ou : cargo install rtk
# Initialisation dans un projet (pose le hook automatiquement) rtk init
```

Économies Mesurées

Commande	Sans RTK	Avec RTK	Réduction
git log	13 994 chars	1 076 chars	92%
git status	100 chars	24 chars	76%
vitest run	~50 000 chars	~5 000 chars	90%
pnpm list	~8 000 chars	~2 400 chars	70%

Une session de 30 minutes avec 10-15 commandes git passe de ~150K à ~41K tokens, soit 72% d'économie sur ce poste seul.

Commandes RTK Essentielles

```
rtk git status # Git condensé rtk
git log # Log sans bruit rtk
cargo test # Tests Rust filtrés rtk
vitest run # Tests JS condensés rtk
gh pr view 123 # PR GitHub optimisé rtk
gain # Dashboard d'économies
```

Compact : Libérer le Contexte

`/compact` résume l'historique de la conversation et libère environ 40-50% du contexte utilisé, sans perdre le fil de la session. À utiliser proactivement à 70% d'utilisation, pas en urgence à 90%.

```
/compact # Résumer la session en cours
```

La différence avec `/clear` : compact conserve la continuité de la session, clear repart de zéro.

.claudeignore : Exclure le Bruit

```
node_modules/ .next/ dist/ *.lock coverage/
```

Sans ce fichier, Claude peut lire des milliers de fichiers non pertinents lors d'une recherche globale, consommant du contexte inutilement.

Règles de Prompt Efficaces

Cibler, pas charger : - «Check the `login` function in

`auth.ts:45-60` » plutôt que «Check auth.ts for issues» -

Référencer des symboles (`calculateTotal`) plutôt que des fichiers entiers - Éviter de coller de gros JSON ou logs bruts dans le prompt

Ce qui consomme le plus :

Action	Tokens estimés
Lecture d'un gros fichier (2000 lignes)	~5K+
Appel MCP (Serena, Context7)	~2K par appel
Long conversation sans /compact	Accumulation
Tests avec output complet non filtré	3K-10K

Stratégie Combinée

RTK réduit le coût des commandes bash (préventif). `/compact`

libère le contexte en cours de session (curatif). `.claudeignore`

évite les lectures inutiles (structurel). Ces trois leviers sont complémentaires et s'appliquent en parallèle.